clase8

October 21, 2026

1 Carga de las librerias

[20]:
```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import MinMaxScaler
```

2 Carga del Dataset

[5]:
```
df = pd.read_csv("../Dataset/dataset_segmentacion_clientes.csv")
```

3 Guardamos unicamente las columnas pertinetes para el modelo

[9]:
```
features = [
    'purchase_freq_month',
    'avg_basket_usd',
    'time_on_site_min',
    'discount_sensitivity',
    'returns_rate',
]
```

4 Chequeamos por valores faltantes

[10]:
```
missing_values = df[features].isnull().sum()
print(missing_values)

if missing_values.sum() == 0:
  print('\nRESULTADO: No hay valores faltantes en el dataset.\n')
else:
  print('\nRESULTADO: Se encontraron valores faltantes.\n')
```

```
purchase_freq_month     0
avg_basket_usd          0
time_on_site_min        0
```

```
discount_sensitivity    0
returns_rate            0
dtype: int64

RESULTADO: No hay valores faltantes en el dataset.
```

5 Chequeamnos si existen outliers usando Turkey

[11]:
```
outliers_summary = {}

for col in features:
  Q1 = df[col].quantile(0.25)
  Q3 = df[col].quantile(0.75)
  IQR = Q3 - Q1
  lower_bound = Q1 - 1.5 * IQR
  upper_bound = Q3 + 1.5 * IQR

  count = len(
      df[(df[col] < lower_bound) | (df[col] > upper_bound)]
  )
  outliers_summary[col] = {
      'Q1': Q1,
      'Q3': Q3,
      'IQR': IQR,
      'Outliers': count,
  }

pd.DataFrame(outliers_summary)
```

[11]:

	purchase_freq_month	avg_basket_usd	time_on_site_min	\
Q1	2.278418	90.766945	6.60395	
Q3	11.214919	449.719033	34.34136	
IQR	8.936501	358.952087	27.73741	
Outliers	0.000000	0.000000	0.00000	

	discount_sensitivity	returns_rate
Q1	0.0	0.0
Q3	1.0	0.8
IQR	1.0	0.8
Outliers	0.0	0.0

5.1 Al no encotrar outliers se decide utilizar MinMaxScaler como escalador

6 Definimos X y aplicamos el escalador a las columnas

[14]:
```
X = df[features]

# 2. Aplicar MinMaxScaler (justificado por la ausencia de outliers)
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

7 Utilizamos el metodo del codo para encontrar el mejor K

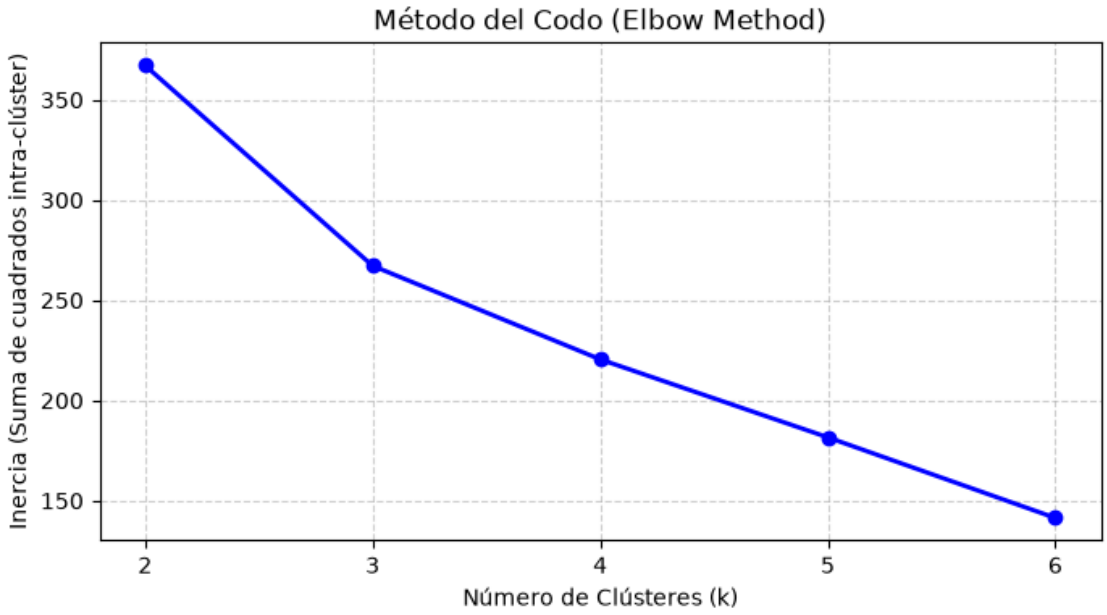
[16]:
```
import os
import warnings

# Silenciar las advertencias de Scikit-Learn y Joblib/Loky
warnings.filterwarnings('ignore')
os.environ['OMP_NUM_THREADS'] = '1'

inertias = []
k_range = range(2, 7)

for k in k_range:
  kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
  kmeans.fit(X_scaled)
  inertias.append(kmeans.inertia_)

# 4. Graficar los resultados directamente en Jupyter
plt.figure(figsize=(8, 4))
plt.plot(k_range, inertias, marker='o', linestyle='-', color='b', linewidth=2)
plt.title('Método del Codo (Elbow Method)')
plt.xlabel('Número de Clústeres (k)')
plt.ylabel('Inercia (Suma de cuadrados intra-clúster)')
plt.xticks(k_range)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



[18]:
```
k_optimo = 3

# 2. Instanciar y ajustar K-Means con los datos ya escalados (X_scaled)
kmeans = KMeans(n_clusters=k_optimo, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

# 3. Guardar las etiquetas de los clústeres en tu DataFrame original
df['cluster'] = clusters

# 4. Calcular el centroide (promedio real de cada variable por clúster)
# Invertimos el escalado para ver los valores en sus unidades originales␣
 ↪(dólares, minutos, etc.)
centroides_escalados = kmeans.cluster_centers_
centroides_originales = scaler.inverse_transform(centroides_escalados)

# Mostrar los centroides en una tabla clara
df_centroides = pd.DataFrame(centroides_originales, columns=features)
df_centroides['tamano_cluster'] = [
    (df['cluster'] == i).sum() for i in range(k_optimo)
]
df_centroides.round(2)
```

[18]:

	purchase_freq_month	avg_basket_usd	time_on_site_min	\
0	11.96	180.00	35.02	
1	1.85	450.12	5.97	
2	4.09	89.96	10.00	

	discount_sensitivity	returns_rate	tamano_cluster
0	0.54	0.33	230
1	0.36	0.38	250
2	0.38	0.30	220

[21]:
```
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=df,
    x='purchase_freq_month',
    y='avg_basket_usd',
    hue='cluster',
    palette='viridis',
    s=60,
    alpha=0.8,
)
plt.title('Segmentación de Clientes por K-Means (k=3)')
plt.xlabel('Frecuencia de Compra Mensual')
plt.ylabel('Ticket Promedio (USD)')
plt.legend(title='Clúster')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

